



# SAP Application Development (S/4 HANA)

HTML5, CSS & JavaScript

## Motivation

This chapter explains how an initial web application using solely HTML5, CSS and JavaScript will be developed. Therefore, you will learn how a basic web application is structured, how its layout can be designed and how the application logic can be implemented.

## Content

This chapter explains how an initial web application simulating a Rock-Paper-Scissors game can be implemented. For this, first a new web application project will be created in Eclipse. Afterwards, a static HTML5 page will be implemented, followed by the creation of a CSS stylesheet which will define the layout of the HTML5 page. Finally, JavaScript functions will be implemented, which determines the logic of the web application.

## Prerequisites

Before starting the exercises, you should have fundamental programming skills and be familiar with the basic navigation in the Eclipse IDE.

## Lecture notes

This chapter is independent from any SAP system. Therefore, students do not need system access for this exercise. However, it is recommended that they have a current web browser installed. Furthermore, this chapter deviates from the flight example which is used in most of the other exercises, but focusses on the development of a small web-based game.

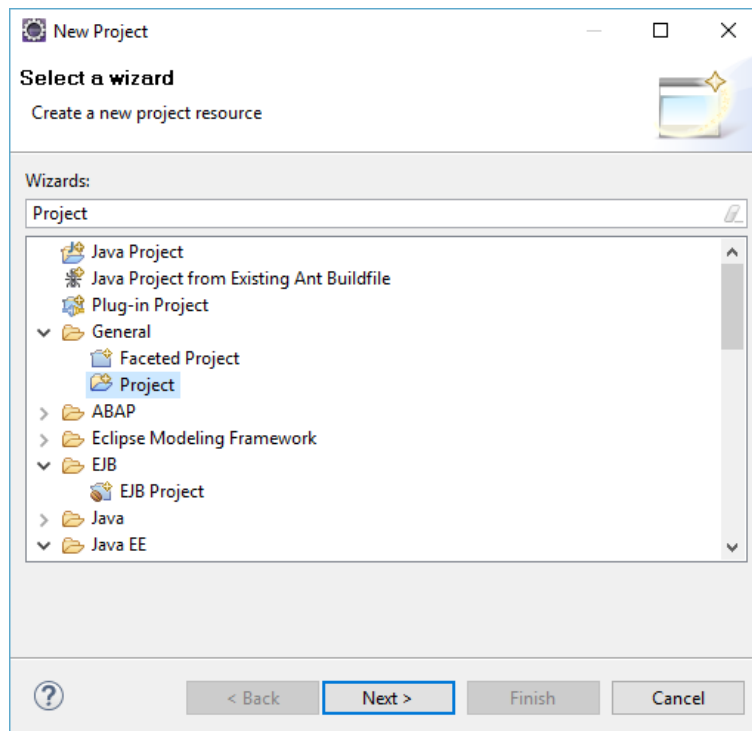
- **Product:** All
- **Level:** Beginner
- **Focus:** Programming
- **Version:** 2.0
- **Author:** UCC Technical University of Munich

**Task 1: Create Web Application Project**

**Short description:** Creating a new web project, which contains HTML5, CSS and JavaScript files

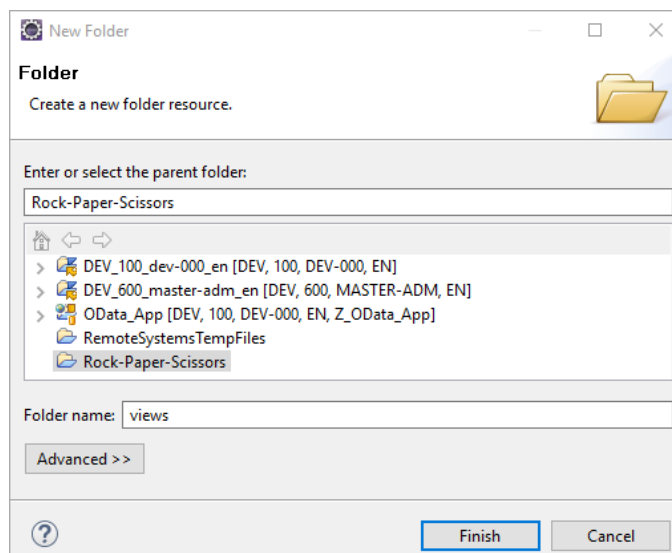
In the context menu of the Project Explorer, choose **New->Other->General->Project**.

*Create new Project*



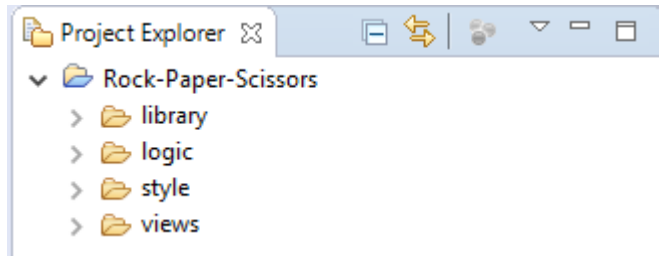
On the next screen, name your project **Rock-Paper-Scissors** (no need to add your 2-digit number ##, since you are developing locally in this exercise), and let all other fields as they are. Click on **Finish**.

Your initially empty project will now appear in the Project Explorer. Since we want to develop a working web application with HTML5, CSS and JavaScript in this exercise, a suitable project structure is needed to keep an overall view of the source files. Therefore, four folders shall be created for the different source files. In the context menu of your project, choose **New->Folder**, and enter Folder Name **views**, which shall contain your HTML5 files later. Click **Finish**.

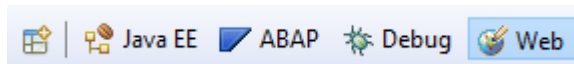


*Create Folders*

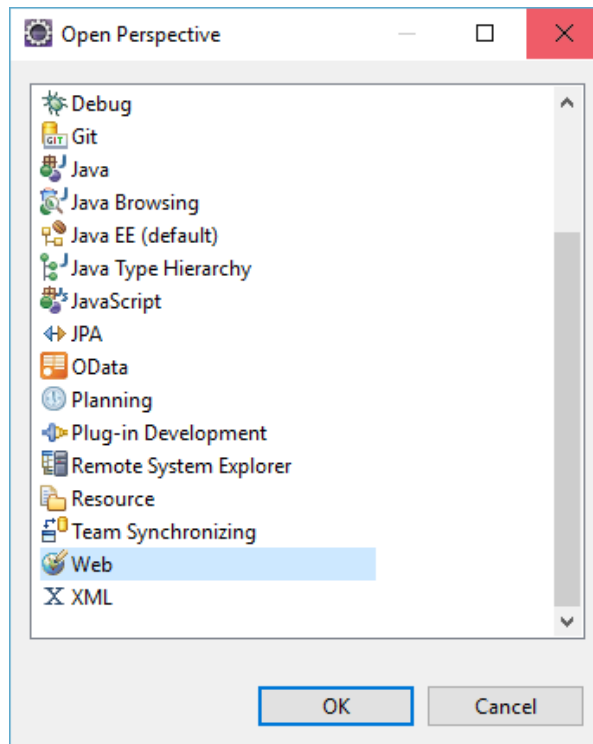
Repeat this step three times, and create the folders **style**, which will contain your CSS files, **logic**, which will contain your JavaScript files, and **library**, which will contain your libraries later.



Finally, since we want to develop a web application, make sure that the **Web** perspective is opened in Eclipse. You can check this on the top right of the UI.



If the **Web** perspective is not yet available there, choose **Window->Perspective->Open Perspective->Other**, and add it to your perspective selection.

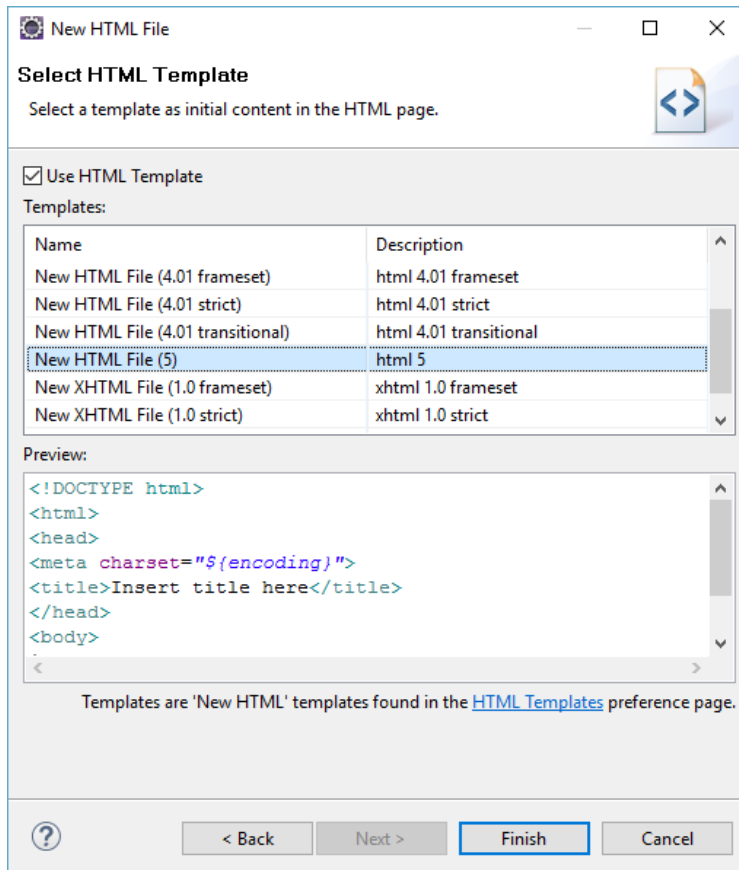


*Open Web  
Perspective*

## Task 2: Create a static HTML5 page

**Short description:** Creating a static HTML5 page which represents a User Interface to play Rock-Paper-Scissors

In the context menu of your **views** folder, select **New->Other->Web->HTML file**, enter File name **play.html**, and click on **Next**. There, select to use an HTML template, and make sure that **New HTML File (5)** is selected. Afterwards, click **Finish**.

**Create HTML5  
file**

An initial template for your HTML file will be created.

```
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
<title>Insert title here</title>
</head>
<body>

</body>
</html>
```

In the next step, we want to create the static elements which will be shown to the user in the browser later. First, enter **Rock-Paper-Scissors** as title between the <title>-tags. Afterwards, add a headline <h1> in the body section with the text **Welcome to Rock-Paper-Scissors**, and a sub-headline <h3> with the description text **Make your choice and beat the computer in multiple rounds!**

```
<title>Rock-Paper-Scissors</title>
</head>
<body>
<h1>Welcome to Rock-Paper-Scissors</h1>
<h3>Make your choice and beat the computer in multiple rounds!</h3>
</body>
```

Next, a fieldset shall be created with which you can select the game mode and make your selection whether you choose rock, paper, or scissors. For this purpose, you have to create different fields, and a list which shows the fields in a structured way.

Furthermore, the details of the different input fields have to be determined. For example, for the game mode, we want to create a dropdown menu. For the Rock-Paper-Scissors selection, we want to use three radio buttons, from which only one can be selected. Altogether, an input button shall start the game with the chosen selection. To implement all these functionalities, add the following program code to the body section of the **play.html** page:

```
<fieldset id="choice">
  <ol>
    <li><label for="mode">Choose your
      mode:&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;</label> <select name="mode" size="1" id="mode">
        <option value="Single Round">Single Round</option>
        <option value="Best of Three" disabled>Best of Three</option>
        <option value="Best of Five" disabled>Best of Five</option>
      </select></li>
    <li><label for="selection">Make your selection:</label> <input
      type="radio" id="r1" name="rps" value="rock" checked>Rock <input
      type="radio" id="r2" name="rps" value="paper">Paper <input
      type="radio" id="r3" name="rps" value="scissors">Scissors</li>
    </ol>
  </fieldset>
  <div>
    <input type="submit" onclick="play()" value="Play!" />
  </div>
```

*Add content to  
HTML5 file*

Thereby, the basic elements of the HTML page were implemented. To test the page in this initial status, save it, right-click on your **play.html** page, and choose **Open With->Web Browser**. A new tab will open in Eclipse, which shows you the current state of your HTML page. It should look like this:

## Welcome to Rock-Paper-Scissors

**Make your choice and beat the computer in up to multiple rounds!**

1. Choose your mode: Single Round ▾

2. Make your selection: ☒ Rock ☐ Paper ☐ Scissors

Play!

*Display initial  
web page*

As you can see, the page currently shows only structured text, but does not contain any style elements. These will follow in the next task, when CSS stylesheets will be included.

To show the results of the Rock-Paper-Scissors game, a dynamic table shall be added to the HTML page, which will be filled with statements whether you won or lost the game. For this purpose, a static table template has to be created, which will only be shown when game results are available. Therefore, please add the following HTML5 code to the body part of the **play.html** file (do not forget to save your file afterwards):

```

<hr>
<div>
  <table border='1' id="results" align="center" style="display: none">
    <thead>
      <tr>
        <th>Results</th>
      </tr>
    </thead>
    <tbody>
    </tbody>
  </table>
</div>

```

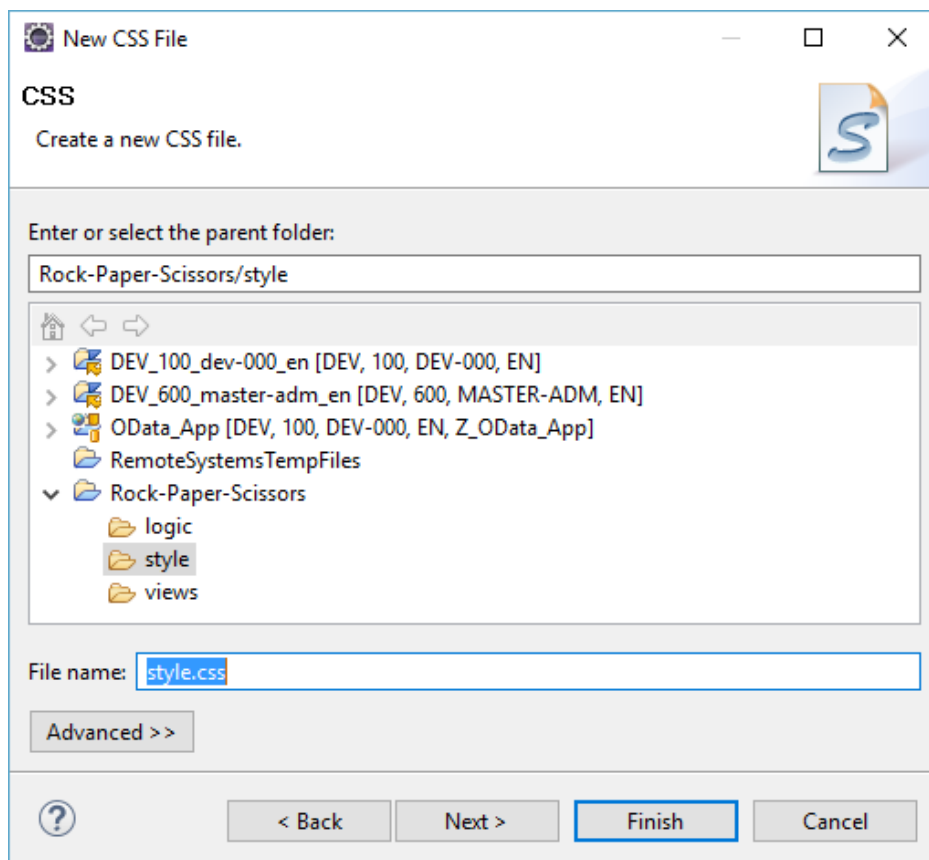
### Task 3: Create CSS Stylesheets

**Short description:** Creating a stylesheet which defines style and layout elements for both HTML pages

In order to define style elements for your HTML page, a central stylesheet file has to be created, which will be included in your HTML file. Therefore, since the style definitions in your CSS file shall be valid for your **play.html** page, first add the following line to its header area, which will include the style definitions:

```
<link href="../../style/style.css" rel="stylesheet" />
```

Afterwards, we have to create the CSS stylesheet. For this purpose, in the context menu of the **style** folder, select **New->Other->Web->CSS file**. Enter the name **style.css** and click on **Finish**.



**Create CSS  
stylesheet**

First, we want to define the background color of the web page, and the style of the shown headlines. For this purpose, add the following code to the CSS file:

```
body {
    background-color: lightsteelblue;
}

h1, h3 {
    text-align: center;
    color: navy;
}
```

*Add style definitions for texts and body*

The **background-color**, respectively **color** attribute simply define the color in which the HTML element shall be displayed. The **text-align** attribute defines how content shall be aligned. In this case, the headlines **h1** and **h3** shall be aligned to the center of the page.

Next, the style of the input fields will be defined. For this, add the following code to the CSS file:

```
fieldset {
    text-align: left;
    background-color: white;
    font-weight: bold;
}

input {
    background-color: navy;
    color: white;
    padding: 15px 32px;
    text-align: center;
    display: inline-block;
    font-size: 20px;
    align: center;
    margin-top: 10px;
}
```

*Add style definitions for input fields*

Again, the text alignment and the color of the elements are defined. Furthermore, the **font-weight** and **font-size** define how text within HTML elements is displayed. Moreover, the **padding** and **margin-top** attributes determine the size and distance of elements.

Finally, the style of the table, which will show the results of the Rock-Paper-Scissors games, is defined. Please add the following code to the CSS file:

```
div {
    text-align: center;
}

table, th, td {
    border: 1px solid grey;
    border-collapse: collapse;
}

th {
    background-color: navy;
    color: white;
    height: 50px;
    width: 400px;
    font-size: 25px;
}

td {
    font-size: 20px;
    height: 50px;
}
```

*Add style definitions for table*

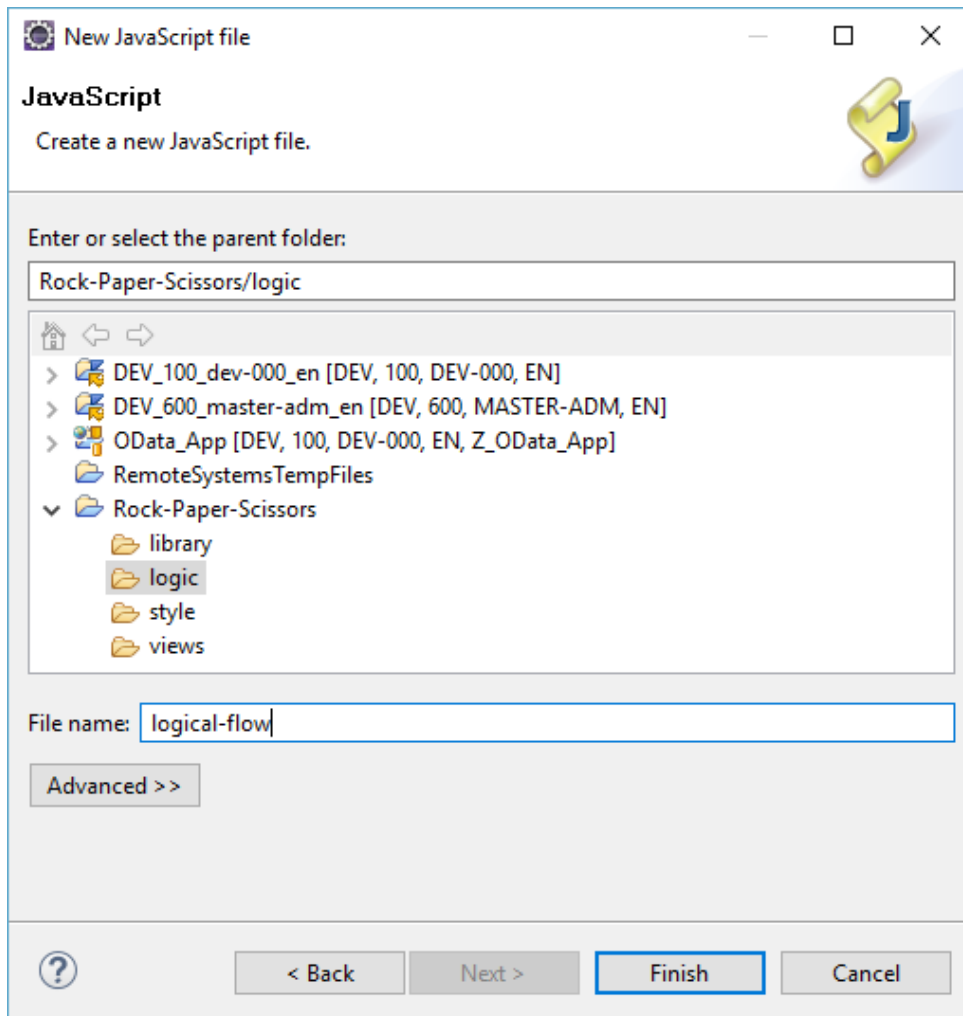
Save your CSS stylesheet. Again, font sizes, colors and text alignments are determined. Furthermore, for the table, the border style and color is defined, as well as the height and width for the table headers.

**Remark:** Manually defining styles in CSS files is a difficult and time-consuming task. In modern web development, this is no longer done manually. For most cases, extensive libraries exist which simplify the style definitions of web pages. This is also the case for developing SAPUI5 applications. However, in this exercise, we shortly demonstrated how manual CSS code is developed, since it is still important to understand how HTML5 and CSS are related.

#### Task 4: Create application logic with JavaScript

**Short description:** Creating a JavaScript file which contains the application logic of the HTML5 page by defining respective functions

To implement the application logic of your web application, first a JavaScript file has to be created which will contain the program code. Therefore, right-click on the **logic** folder of your project, and select **New->Other->JavaScript->JavaScript Source File**. Click on **Next**, enter the name **logical-flow**, and click on **Finish**.



*Create  
JavaScript file*

Since we want to use jQuery to simplify the manipulation of HTML5 objects, please go to <http://jquery.com/download/>, and download the current version of the compressed jQuery source file. Rename the downloaded file to **jquery.js**, and add it to the **library** folder of your project.



Next, we want to include the downloaded jQuery library, as well as the created **logical-flow.js** file. Therefore, please go to your **play.html** page, and add the following lines to the header section (do not forget to save afterwards):

```
<script src="../../logic/logical-flow.js" type="text/javascript"></script>
<script src="../../library/jquery.js" type="text/javascript"></script>
```

Now, we are prepared to implement the application logic using JavaScript and jQuery. As you may have probably noticed, in the **play.html** file, we used the **on-click()**-event of the input button to call the function **play()**. However, we did not implement this function so far. Therefore, we want to add it to our JavaScript file. The function shall read the mode the user selected for his game, and call additional functions to play the game and show the results. Hence, add the following code to your **logical-flow.js** file:

```
function play() {

    // Read mode
    var mode = document.getElementById("mode");
    var selectedMode = mode.options[mode.selectedIndex].value;

    if (selectedMode == "Single Round") {

        var gameResult = playRound();
        showResult(gameResult);

    } else {

        // TODO: Implementation of more difficult modes

    }

}
```

*Add JavaScript functionalities*

As implemented, the program code reads the mode and calls the **playRound()** function to play the game. Depending on whether you won or lost the game, the **showResult()**-function shows a respective message. Currently, we only implemented the mode to play a single round, for which reason, the other two modes are not working. Implementing one of these modes will be part of the challenge which follows at the end of this exercise.

The **playRound()** function contains the central logic of the web application with regard to computing which player won the game. This function first reads the selection of Player 1, which is the one using the web application. Afterwards, it computes a random number between 1 and 3, which determines whether Player 2 (the computer) chooses rock, paper, or scissors. Finally, the selections of Player 1 and 2 are compared and the winner is determined. To implement these functionalities, please add the following JavaScript code to your file:

```
function playRound() {

    // Read selection of Player 1
    var player1;
    if ($('#r1').is(':checked')) {
        player1 = "Rock";
    } else if ($('#r2').is(':checked')) {
        player1 = "Paper";
    } else if ($('#r3').is(':checked')) {
        player1 = "Scissors";
    }

}
```

```

// Compute random choice of Player 2
var min = 1;
var max = 4;
var random = Math.floor(Math.random() * (max - min)) + min;
var player2;
switch (random) {
case 1:
    player2 = "Rock";
    break;
case 2:
    player2 = "Paper";
    break;
case 3:
    player2 = "Scissors";
    break;
default:
    alert("Error");
}

var result = "";
// Determine result
if (player1 == player2) {
    result = "Both players have made the same choice! Play again!";
} else if (player1 == "Rock" && player2 == "Paper") {
    result = "Opponent has chosen... PAPER! Paper beats Rock! You lost this round!";
} else if (player1 == "Rock" && player2 == "Scissors") {
    result = "Opponent has chosen... SCISSORS! Rock beats Scissors! You won this round!";
} else if (player1 == "Paper" && player2 == "Scissors") {
    result = "Opponent has chosen... SCISSORS! Scissors beat Paper! You lost this round!";
} else if (player1 == "Paper" && player2 == "Rock") {
    result = "Opponent has chosen... ROCK! Paper beats Rock! You won this round!";
} else if (player1 == "Scissors" && player2 == "Paper") {
    result = "Opponent has chosen... PAPER! Scissors beat Paper! You won this round!";
} else if (player1 == "Scissors" && player2 == "Rock") {
    result = "Opponent has chosen... ROCK! Rock beats Scissors! You lost this round!";
}

return result;
}

```

Finally, the result determined by the **playRound()**-function shall be shown to the user. For this purpose, the table template defined in the **play.html** file has to be extended and displayed to the user. Therefore, please add the following function to your JavaScript file:

```

function showResult(result) {

    // Append html element to show result
    $('#results').show();
    $('#results').append('<tr><td>' + result + '</td></tr>');

}

```

**Manipulate  
DOM**

Save your JavaScript file. Finally, your web application has been implemented. If you already want to test it, right-click on your **play.html** file, and choose **Open With->Web Browser**. If you implemented all steps correctly, the web page will be shown with your style definitions, and by clicking on play, the results of the game will be shown in a table. You can also copy the link of your local **play.html** file, which is shown at the top of the integrated browser, into a local web browser (e.g. Chrome). This way, you can inspect the source code with the integrated developer tools of your browser.

## Welcome to Rock-Paper-Scissors

Make your choice and beat the computer in up to multiple rounds!

1. Choose your mode: Single Round ▾

2. Make your selection: ☐ Rock ☐ Paper ☒ Scissors

Play!

| Results                                                                    |
|----------------------------------------------------------------------------|
| Opponent has chosen... PAPER! Paper beats Rock! You lost this round!       |
| Opponent has chosen... PAPER! Scissors beat Paper! You won this round!     |
| Both players have made the same choice! Play again!                        |
| Opponent has chosen... ROCK! Paper beats Rock! You won this round!         |
| Opponent has chosen... ROCK! Paper beats Rock! You won this round!         |
| Opponent has chosen... SCISSORS! Scissors beat Paper! You lost this round! |
| Opponent has chosen... PAPER! Scissors beat Paper! You won this round!     |

### Challenge: Implement the additional mode “Best of Three”

**Task 1:** Currently, if a user selects another mode except **Single Round**, nothing happens. Please extend the game by implementing the **Best of Three** mode. Extend the **play** method by reacting to the event if a user chooses the **Best of Three** mode. In this case, call the **playRound** function as it is done for the **Single Round** mode. However, after every round, further add the following text to the table: “**Note: You have chosen Best of Three mode. Please evaluate whether one of the players already won the game!**”

**Task 2 (difficult!):** Collect information about cookies in JavaScript, e.g. on [http://www.w3schools.com/js/js\\_cookies.asp](http://www.w3schools.com/js/js_cookies.asp). Implement a counter with cookies, which counts the number of rounds played in the current **Best of Three** game. To simplify the rules, assume that every **Best of Three** game has three rounds, even if one player has already won after round 2. In the **play** function, read the current round number from the cookie. If you have already reached round 3 (and only in this case), add the following text to the table: “**Three rounds were played in the Best of Three mode. The game is over. Please evaluate who won the game!**”